

Problem Solving Using Python (CSE1012)

Dr. Abdul Kayom, Assistant Professor, SENSE

Agenda / Contents

01 Introduction to Functions

02 Need for Functions

03 Function Definition

04 Function Call

05

Variable Scope and Lifetime

- You can't just access any variable from any part of your program
- Some variables may not even exist for the entire duration of the program
- In which part of the program you can access a variable and in which parts of the program a variable exists depends on- how the variable has been declared

Variable Scope and Lifetime

- To determine the scope and lifetime of a variable, we need to understand these two things:
 - Scope: part of the program in which a variable is accessible
 - Lifetime: duration for which the variable exists

Local and Global Variables

- Global variables are defined in the main body of the program file
- Accessible throughout the program
- You should be very economical with variables: define global variable if it is meant to be used globally

Local and Global Variables

- A variable defined within a function is local to it
- It is accessible within the function
- It exists as long as the function is executing
- Function parameters behave like local variables in the function

Local and Global Variables

#Example to demonstrate local & global variables

```
num1 = 10
```

```
num2 = 20
```

```
def add_num(n1, n2)
```

```
    num3 = n1 + n2
```

```
    return num3
```

```
SUM = add_num(num1, num2)
```

```
print(SUM)
```

```
print(num3)      #NameError: name 'num3' is not defined
```

Local and Global Variables

Table 5.1 Comparison Between Global and Local Variables

Global Variables	Local Variables
1. They are defined in the main body of the program file.	1. They are defined within a function and is <i>local</i> to that function.
2. They can be accessed throughout the program file.	2. They can be accessed from the point of its definition until the end of the block in which it is defined.
3. Global variables are accessible to all functions in the program.	3. They are not related in any way to other variables with the same names used outside the function.

Using the Global Statement

- To define a global variable inside a function, you must use the global statement
- This declares the local or the inner variable of the function to have module scope

Using the Global Statement

#Using global statement to declare a global variable inside function

```
str1 = "Good "
```

```
def show():
```

```
    global str2
```

```
    str2 = "Morning"
```

```
show()
```

```
str3 = str1 + str2
```

```
print(str3)
```

Same Variable Name in and outside

#Using same Variable Name inside & Outside

```
var = "Morning "
```

```
def show():
```

```
    var = "Good"
```

```
    print("Inside Function, var is: ",var)
```

```
show()
```

```
print("Outside Function, var is: ",var)
```

Same Variable Name in and outside

#Demonstrate Same Name of Variable in and outside of a function

```
var = "Good"
```

```
def show():
```

```
    global var
```

```
    var = "Morning"
```

```
    print("Inside Function, Var is: ",var)
```

```
show()
```

```
print("Outside Function, Var is: ",var)
```

```
var = "Fantastic"
```

```
print("Outside Function, after modification, Var is: ",var)
```

The return Statement

- Even if there is no explicit return statement inside the function, every function has an implicit return statement
- By default it returns nothing to the calling function
- But you can change this behavior with an explicit return statement, when the function is returning some value

The return Statement

- The return statement must be inside the function
- Once you return a value from a function, it immediately exits that function
- Any code written after return statement is never executed

The return Statement

#Demonstrate the 'return' statement

```
def display():
```

```
    print("Inside Function")
```

```
    print("About to Execute the return statement")
```

```
    return
```

```
    print("This line will never be displayed")
```

```
display()
```

```
print("Back to the caller")
```

Required Arguments

- Arguments are passed to a function in correct order
- Also number of arguments in the function call should exactly match with the number of arguments specified in the function

Required Arguments

#Example1 of Incorrect Argument Pass

```
def display():
```

```
    print("Hello")
```

```
display("Hi")
```

Required Arguments

#Example2 of Incorrect Argument Pass

```
def display(str):
```

```
    str="Hello"
```

```
    print(str)
```

```
display()
```

Required Arguments

#Example of Correct Argument Pass

```
def display(str):
```

```
    print(str)
```

```
str="Hello"
```

```
display(str)
```

Default Arguments

- Python allows users to specify function arguments that can have default values
- i.e. a function can be called with fewer arguments than it is defined to have (the argument not specified must have predefined default value)

Default Arguments

#Example of Default Value of Argument

```
def display(name, course = "Btech"):
```

```
    print("Name is: ",name)
```

```
    print("Course is: ",course)
```

```
display(name = "Arnav")
```

Default Arguments

#Example of Default Value of Argument

```
def display(name, course = "Btech"):
```

```
    print("Name: ",name)
```

```
    print("Course: ",course)
```

```
display(course = "Btech")
```

#it will give error.

#because, essential argument

#name is not passed

Thank You

Dr. Abdul Kayom



Asst. Prof, (SENSE)



kayomabdul@vitap.ac.in



8474860025